

Python – Ejercicios Extra

Equipo: Formación Python

Versión del documento: 1.2

Control de cambios

Versión	Fecha	Descripción cambios	Autor
1.0	1 feb 2025	Creación del documento	Jose DelPino Cas...
1.1	15 oct 20...	Aplicación del nuevo rebranding de Nter	Antonio José Tira...
1.2	17 feb 20...	Mejora y ajuste de los enunciados de los ejercicios	Antonio José Tira...

Índice.

Control de cambios	1
Índice.	2
Ejercicio 1	3
Ejemplo de ejecución	3
Ejercicio 2	3
Ejemplo de ejecución	3
Ejercicio 3	4
Ejemplo de ejecución	4
Ejercicio 4	5
Ejemplo de ejecución	5
Ejercicio 5	5
Ejemplo de ejecución	5
Ejercicio 6	6
Ejemplo de ejecución	6
Ejercicio 7	7
Ejemplo de ejecución	7
Ejercicio 8	8
Ejemplo de ejecución	8
Ejercicio 9	8
Ejemplo de ejecución	9
Ejercicio 10	10
Ejemplo de ejecución	10
Ejercicio 11	10
Ejercicio 12	12
Contacto	13

Ejercicio 1

Escribe un programa que pida al usuario **dos números enteros**. A continuación, el programa debe calcular la **división entera** del primer número entre el segundo y mostrar por pantalla el resultado utilizando **exactamente el siguiente formato**:

"<n> entre <m> da un cociente <c> y un resto <r>"

Donde <n> y <m> son los números introducidos por el usuario, y <c> y <r> son el cociente y el resto de la división entera, respectivamente.

Ejemplo de ejecución

- **Entrada:** 27 y 4
- **Salida:** 27 entre 4 da un cociente 6 y un resto 3

Ejercicio 2

Escribe un programa que pida al usuario su fecha de nacimiento utilizando el formato **dd/mm/aaaa**. A continuación, el programa debe extraer y mostrar por pantalla el día, el mes y el año por separado.

Importante: El programa debe funcionar correctamente tanto si el usuario introduce el día y el mes con **dos dígitos** (ej. 20/02/2000) como si lo hace con **un solo dígito** (ej. 20/2/2000 o 5/8/1995).

Ejemplo de ejecución

- **Entrada:** 02/02/2000 o 2/2/2000
- **Salida:** Día: 02 | Mes: 02 | Año: 2000

Ejercicio 3

En una determinada empresa, los empleados son evaluados al final de cada año. Los puntos que pueden obtener en esta evaluación determinan su nivel de rendimiento y el dinero de su bonificación anual.

Las puntuaciones válidas **solo pueden ser 0.0, 0.4, 0.6 o cualquier valor superior a 0.6.**

Importante: No existen valores intermedios entre estas cifras (por ejemplo, una puntuación de 0.2 o 0.5 no es válida, y el programa debe avisar al usuario de este error).

A continuación se muestra la tabla con los niveles correspondientes a cada puntuación:

Nivel	Puntuación
Inaceptable	0.0
Aceptable	0.4
Meritorio	0.6 o más

La cantidad de dinero conseguida en cada nivel se calcula multiplicando una base de **2.400€ por la puntuación** obtenida.

Escribe un programa que pida al usuario su puntuación y muestre por pantalla su **nivel de rendimiento**, así como la **cantidad de dinero** que recibirá.

Ejemplo de ejecución

- **Entrada A:** 0.4
- **Salida A:** Tu nivel de rendimiento es: **Aceptable**. Te corresponde una bonificación de: **960.0€**
- **Entrada B:** 0.5
- **Salida B:** Error: **Puntuación inválida**. Los valores permitidos son **0.0, 0.4, 0.6 o superiores**.

Ejercicio 4

Escribe un programa que almacene las siguientes dos matrices en variables y muestre por pantalla el resultado de su producto (multiplicación matricial fila por columna).

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad y \quad B = \begin{pmatrix} -1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$$

Nota importante: Para representar las matrices, debes usar estrictamente **listas anidadas**, donde cada lista interna representa un vector fila de la matriz (por ejemplo, $A = [[1, 2, 3], [4, 5, 6]]$). El cálculo debe realizarse mediante programación clásica (usando bucles), sin importar librerías externas.

Ejemplo de ejecución

- **Salida:** El producto de las matrices A y B es: $[[2, 5], [2, 11]]$

Ejercicio 5

Escribe un programa que almacene los siguientes precios en una **lista**: $[50, 75, 46, 22, 80, 65, 8]$.

A continuación, el programa debe analizar la lista y mostrar por pantalla cuál es el **menor** y cuál es el **mayor** de todos los precios incluidos en ella.

Nota: Piensa si vas a resolverlo utilizando herramientas integradas del lenguaje o si vas a construir tu propio algoritmo de búsqueda iterando sobre la lista).

Ejemplo de ejecución

- **Salida:** El precio menor es **8** y el precio mayor es **80**.

Ejercicio 6

El directorio de clientes de una empresa está organizado en una única cadena de texto. Las líneas de esta cadena están separadas por el carácter de salto de línea (`\n`), y la primera línea contiene los nombres de las columnas. Dentro de cada línea, los datos están separados por punto y coma (`;`).

A continuación se muestra la cadena de texto que debes almacenar en una variable:

```
"nif;nombre;email;teléfono;descuento\n01234567L;Luis  
González;luisgonzalez@mail.com;656343576;12.5\n71476342J;Macarena  
Ramírez;macarena@mail.com;692839321;8\n63823376M;Juan José  
Martínez;juanjo@mail.com;664888233;5.2\n98376547F;Carmen  
Sánchez;carmen@mail.com;667677855;15.7"
```

Escribe un programa que procese esta cadena y genere un **diccionario anidado** (un diccionario que contiene otros diccionarios) con la información del directorio.

Estructura del diccionario resultante:

- Cada elemento principal corresponde a un cliente.
- La **clave** principal debe ser su `nif`.
- El **valor** principal debe ser **otro diccionario** con el resto de su información. Las claves de este sub-diccionario serán los nombres de los campos de la primera línea (`nombre`, `email`, `teléfono`, `descuento`).

Nota: Presta atención al campo "descuento". En el resultado final debería tratarse como un número decimal (float), no como texto.

Ejemplo de ejecución

- **Salida:** El programa debe imprimir un diccionario exactamente (o muy similar) a este:

```
{  
    '01234567L': {'nombre': 'Luis González', 'email':  
'luisgonzalez@mail.com', 'teléfono': '656343576', 'descuento': 12.5},  
    '71476342J': {'nombre': 'Macarena Ramírez', 'email':  
'macarena@mail.com', 'teléfono': '692839321', 'descuento': 8.0},  
    '63823376M': {'nombre': 'Juan José Martínez', 'email':  
'juanjo@mail.com', 'teléfono': '664888233', 'descuento': 5.2},  
    '98376547F': {'nombre': 'Carmen Sánchez', 'email': 'carmen@mail.com',  
'teléfono': '667677855', 'descuento': 15.7}  
}
```

Ejercicio 7

Escribe un programa que estructure su código utilizando las siguientes **dos funciones**:

1. **Primera función:** Debe recibir una cadena de caracteres (texto) y devolver un **diccionario** con cada palabra que contiene y su frecuencia (el número de veces que aparece). *Nota: Recuerda convertir todo el texto a minúsculas antes de contar, para que palabras como "Hola" y "hola" se cuentan como la misma*.
2. **Segunda función:** Debe recibir el diccionario generado por la función anterior y devolver una **tupla** que contenga la palabra más repetida y su frecuencia, en este formato: (palabra, frecuencia).

Ejemplo de ejecución

- **Entrada:** "Hola como estas hola muy bien hola estas"
- **Salida 1:** { 'hola': 3, 'como': 1, 'estas': 2, 'muy': 1, 'bien': 1 }
- **Salida 2:** ('hola', 3)

Ejercicio 8

Escribe un programa que incluya las siguientes **dos funciones**:

1. **De decimal a binario:** Una función que reciba un número entero decimal y devuelva una **cadena de texto** con su representación en binario.
2. **De binario a decimal:** Una función que reciba una cadena de texto que represente un número binario y devuelva su valor **entero** en decimal.

Nota importante: Para este ejercicio, **no** está permitido utilizar las funciones integradas de conversión de Python (como `bin()` o la conversión base 2 de `int()`). Debes programar la lógica matemática utilizando bucles (divisiones sucesivas entre 2 para el primer caso, y potencias de 2 para el segundo).

Ejemplo de ejecución

- **Salida 1:** Al ejecutar la primera función con el número 19, el resultado debe ser: "10011"
- **Salida 2:** Al ejecutar la segunda función con la cadena "10011", el resultado debe ser: 19

Ejercicio 9

Una inmobiliaria maneja su cartera de viviendas utilizando una lista de diccionarios. Cada diccionario contiene los datos de un inmueble:

```
inmuebles = [  
    {'año': 2001, 'metros': 100, 'habitaciones': 3, 'garaje': True, 'zona': 'A'},  
    {'año': 2012, 'metros': 60, 'habitaciones': 2, 'garaje': True, 'zona': 'B'},  
    {'año': 1980, 'metros': 120, 'habitaciones': 4, 'garaje': False, 'zona': 'A'},  
    {'año': 2005, 'metros': 75, 'habitaciones': 3, 'garaje': True, 'zona': 'B'},  
    {'año': 2015, 'metros': 90, 'habitaciones': 2, 'garaje': False, 'zona': 'A'}  
]
```

Construye una función que permita buscar inmuebles en función de un presupuesto máximo. La función debe cumplir las siguientes condiciones:

1. **Entrada:** Debe recibir dos parámetros: la lista de inmuebles y un número con el presupuesto máximo.
2. **Cálculo del precio:** Para cada inmueble, debe calcular su precio en función de su zona. La *antigüedad* se calcula restando el año de construcción al **año actual** (**asume que es 2026**).
 - a. **Fórmula para Zona A:**
 - i.
$$\text{Precio} = (\text{metros} * 1000 + \text{habitaciones} * 5000 + \text{garaje} * 15000) * (1 - \text{antigüedad} / 100)$$
 - b. **Fórmula para Zona B:**
 - i.
$$\text{Precio} = (\text{metros} * 1000 + \text{habitaciones} * 5000 + \text{garaje} * 15000) * (1 - \text{antigüedad} / 100) * 1.5$$
3. **Actualización de datos:** Debe añadir una nueva clave '**precio**' a los diccionarios de los inmuebles con el valor calculado.
4. **Salida:** Debe devolver **una lista nueva** que contenga únicamente los inmuebles cuyo precio sea **menor o igual** al presupuesto dado.

*Nota: Recuerda que en Python, el valor booleano **True** equivale numéricamente a **1** y **False** equivale a **0**. ¡Puedes usar la variable **garaje** directamente en la multiplicación de la fórmula!*

Ejemplo de ejecución

- **Salida:** Si ejecutamos la función pasando la lista anterior y un presupuesto de **100000**, la lista devuelta debería ser:

```
[  
  
    {'año': 2001, 'metros': 100, 'habitaciones': 3, 'garaje': True,  
     'zona': 'A', 'precio': 96200.0},  
  
    {'año': 2015, 'metros': 90, 'habitaciones': 2, 'garaje': False,  
     'zona': 'A', 'precio': 89100.0}  
]
```

Ejercicio 10

Escribe una función que reciba una lista de números (una muestra) y devuelva una **nueva lista** que contenga únicamente los valores atípicos.

Un valor se considera atípico si su **puntuación típica (z-score)** es estrictamente **mayor que 3 o menor que -3**.

Nota importante: La puntuación típica de cada valor se obtiene restando la media de la muestra al valor, y dividiendo el resultado por la desviación típica de la muestra.

1. **Media:** Suma de todos los valores dividida entre la cantidad de valores.
2. **Desviación típica:** La raíz cuadrada de la varianza. (La varianza es la media de los cuadrados de las diferencias entre cada valor y la media).
3. **Fórmula de la puntuación típica:** $z = (\text{valor} - \text{media}) / \text{desviación_típica}$

Nota: Puedes decidir si vas a programar tus propias funciones auxiliares para calcular la media y la desviación típica paso a paso utilizando bucles, o si vas a investigar cómo utilizar el módulo integrado `statistics` o `math` de Python para facilitarte el trabajo.

Ejemplo de ejecución

- **Entrada:** [10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 200]
- **Salida:** [200] (Ya que la puntuación típica de 200 en esa muestra concreta es aproximadamente 3.74, lo cual es mayor que 3).

Ejercicio 11

Escribe un programa para gestionar un listín telefónico con los nombres y los teléfonos de los clientes de una empresa. El listín debe guardarse físicamente en un fichero de texto llamado `listin.txt`.

Reglas de formato del fichero:

- Cada cliente debe ocupar una línea distinta.
- El nombre y el teléfono deben estar separados por una coma (ejemplo: Ana García,666555444).

El programa debe estar estructurado de forma modular, incorporando al menos las siguientes **funciones**:

1. **Crear fichero:** Una función que compruebe si el fichero `listin.txt` existe y, si no existe, lo cree vacío.
2. **Añadir cliente:** Una función que pida un nombre y un teléfono, y los añada al final del fichero respetando el formato.
3. **Consultar cliente:** Una función que reciba el nombre de un cliente, lo busque en el fichero y muestre por pantalla su teléfono. (Si no existe, debe avisar al usuario).
4. **Eliminar cliente:** Una función que reciba el nombre de un cliente y elimine su registro completo (su línea) del fichero.

Nota: Recuerda repasar los modos de apertura de ficheros en Python ('r' para leer, 'w' para escribir desde cero, y 'a' para añadir al final). ¡Ten especial cuidado con la función de eliminar, ya que normalmente requiere leer todo el contenido, filtrar el que no quieres, y volver a sobrescribir el fichero!

Ejemplo visual de cómo debería quedar el archivo `listin.txt` por dentro:

Juan Pérez,612345678

María López,698765432

Carlos Gómez,655444333

Ejercicio 12

El fichero `calificaciones.csv` contiene las notas de los alumnos de un curso. Durante el semestre se realizaron dos exámenes parciales de teoría y un examen de prácticas. Aquellos alumnos que obtuvieron menos de un 4 en alguno de estos exámenes pudieron repetirlo al final del curso en la convocatoria ordinaria.

Ejemplo hipotético de cómo se ven los datos en el fichero CSV:

```
Apellidos;Nombre;Asistencia;Parcial1;Parcial2;Practicas;Ordinaria1;Ordinaria2;OrdinariaPracticas
García;Ana;80%;6.5;3.0;7.0;;5.5;
```

Escribe un programa modular que contenga, al menos, las siguientes **tres funciones**:

1. **Lectura y estructuración:** Una función que reciba el nombre del fichero de calificaciones y devuelva una **lista de diccionarios**. Cada diccionario contendrá la información y notas de un único alumno. **Importante:** La lista devuelta debe estar **ordenada alfabéticamente por apellidos**.
2. **Cálculo de la nota final:** Una función que reciba la lista de diccionarios anterior y añada a cada diccionario un nuevo par clave-valor con la '**Nota Final**' del curso.
 - a. El peso de cada parcial de teoría es de un **30%** (0.3).
 - b. El peso del examen de prácticas es de un **40%** (0.4).
 - c. *Nota: Si un alumno se presentó a la convocatoria ordinaria para recuperar un examen, debes utilizar esa nota para el cálculo en lugar de la nota del parcial original. ¡Cuidado con limpiar los datos si las notas o la asistencia vienen como texto!*
3. **Evaluación de aprobados y suspensos:** Una función que reciba la lista de diccionarios con las notas finales calculadas y devuelva **dos listas separadas**: una con los alumnos aprobados y otra con los suspensos.
 - a. Para que un alumno sea considerado **aprobado**, debe cumplir **todas** estas condiciones a la vez:
 - i. La asistencia debe ser mayor o igual al **75%**.

- ii. La nota de cada uno de los exámenes (parciales y prácticas, o sus respectivas recuperaciones) debe ser mayor o igual a **4**.
- iii. La nota final calculada debe ser mayor o igual a **5**.

Contacto

Si durante la realización de los ejercicios encuentras alguna duda o necesitas aclaraciones, no dudes en contactar con tu formador asignado.

¡Buena suerte!